

What is CSS?

CSS (Cascading Style Sheets) is a stylesheet language used to style and design HTML elements on a web page.

HTML defines the structure and content of a webpage, while CSS controls its appearance and layout.

Ways to Add CSS

CSS can be added to HTML in three ways:

1. Inline CSS

CSS is written directly inside the HTML element using the style attribute for applying a style to a single element.

style="css1: value; css2 : value;"

```
<h1 style="color: blue;">Hello World</h1>
```

2. Internal CSS

CSS is written inside the <style> </style> tag, usually within the <head></head> section for styling a single HTML page.

```
<head>
  <style>
    h1 {
      color: blue;
    }
  </style>
</head>
```

3. External CSS

CSS is written in a separate **.css** file and connected to HTML using the <link> tag.

Best approach for most real-world websites because the same CSS file can be used across multiple HTML pages.

— **index.html (HTML FILE)** —

```
<head>
  <link rel="stylesheet" href="style.css">
</head>
```

— **style.css (CSS FILE)** —

```
h1 {
  color: blue;
}
```

CSS Selectors

A CSS Selector is used to select the HTML element, that we want to style.

Syntax:

```
selector {  
    property: value;  
}
```

Types of CSS Selectors

1. Element Selector :

Element Selector selects all HTML elements of a specific type or tag.

```
ex.:  p {  
        color: blue;  
    }
```

2. ID Selector

ID Selector selects an HTML element using its unique id attribute using (#) symbol.

```
ex.:  #title {  
        color: red;  
    }
```

```
<h1 id="title">Apply red color only in this heading tag</h1>
```

3. Class Selector

Class Selector selects one or more HTML elements having the same class name using (.) symbol.

```
ex.:  .text {  
        color: green;  
    }
```

```
<h1 class="title">Apply red color only in this heading tag</h1>
```

```
<h3 class="title">Apply red color only in this heading tag</h3>
```

4. Universal Selector

Universal Selector selects all HTML elements on the page using (*) symbol.

```
ex.:  * {  
        margin: 0;  
        padding: 0;  
    }
```

5. Grouping Selector

```
h1, h2, p {color: blue; }
```

SOME CSS PROPERTIES**1. CSS Colors**

color : red **or** rgb(255, 255, 0) **or** #FEDA00; */* It is used for coloring the text only*/*

background-color : red **or** rgb(255, 255, 0) **or** #FEDA00; */* It is used for coloring the background/or any elements only*/*

border-color : red **or** rgb(255, 255, 0) **or** #FEDA00; */* It is used for coloring the outline of the box*/*

Color formats: (Name, RGB, RGBA, HEX)

2. CSS Background

Background properties control the color and image displayed (clipping mask image) behind an element.

Properties	—	values

background-color :		red or #ff000 or rgb(255, 0, 255);
background-image :		url(/images/bg-jpg);
background-repeat :		no-repeat or no-repeat or repeat-x or repeat-y;
background-position :		center or left or top or bottom or right or 50% 50%;
background-size :		cover or contain or auto 100% 100%;
background-attachment :		fixed or scroll (default value);
background:		linear-gradient(rgba(0, 0, 0, 0.6), rgba(0, 0, 0, 0.6)), url('your-image.jpg')
no-repeat center/cover;		

3. CSS Text

Text properties control alignment decoration, transformation and spacing etc.

Properties	—	values

text-align :		center or left or right or justify;
text-decoration :		none or underline overline line-through;
text-transform :		uppercase or lowercase or capitalize or none;
line-height :		normal or 2px;
letter-spacing :		normal or 2px;
word-spacing :		normal or 2px;

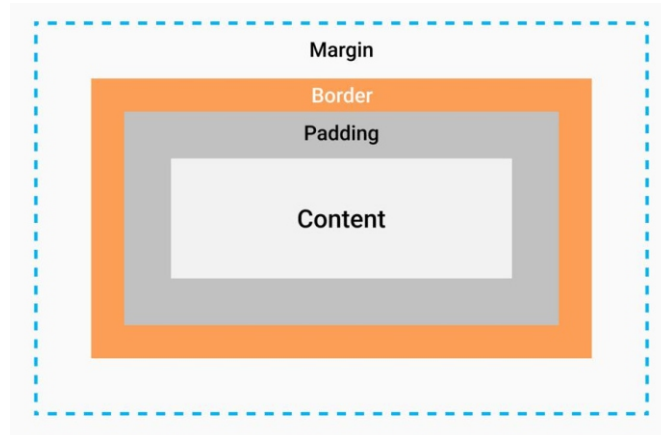
4. box-shadow

For apply a shadow effect on any box element like (any container)

box-shadow : 2px 2px 10px 5px #EAEAE6;

Box Model :

Every HTML element is treated as a rectangular box with these four parts – content, padding, border, and margin. The Box Model describes how an element's total size is formed

**5. CSS Border**

Borders create a visible boundary around an element.

Properties — values

border-style :	solid or dashed or dotted or double or none;
border-width :	2px or thin or medium thick;
border-color :	red or #FED500 or rgb(.....);
border-top :	2px solid red;
border-right :	2px solid red;
border-bottom :	2px solid red;
border-left :	2px solid red;
border :	1px solid orange;

6. CSS Margin & Padding

Margin creates space outside of an element, while Padding creates space inside its element.

Properties — values

margin :	20px; /* Single value for all 4 side in clock wise - top, right, bottom, left)
margin :	10px 20px; /* Two value for 2 by 2 side (first - top bottom , second right left)
margin :	20px 6px 10px 20px; /* for all 4 side in clock wise - top, right, bottom, left)
margin-top / right / bottom / left :	2px or thin or medium thick;
margin :	auto or 0 auto; /* This value is for center an element*/
padding :	20px; /* Single value for all 4 side in clock wise - top, right, bottom, left)
padding :	10px 20px; /* Two value for 2 by 2 side (first - top bottom , second right left)
padding :	20px 6px 10px 20px; /* for all 4 side in clock wise - top, right, bottom, left)
padding-top / right / bottom / left :	2px or thin or medium thick;

7. Display & Visibility :

Display controls how an element behaves and occupies space in the layout.

display : block or inline or inline-block or none;

Visibility: visibility controls whether an element is visible or hidden without changing its layout space.

visibility : visible or hidden;

8. Overflow controls content that goes outside an element's defined area.

overflow : visible or hidden or scroll or auto

overflow-x : visible or hidden or scroll or auto

overflow-y : visible or hidden or scroll or auto

9. CSS Position

Position properties control where an element is placed relative to its normal position or containing element.

position : static or relative or absolute or fixed or sticky

Note : relative will be treated as parent div and absolute will be treated as child of that relative div

top : 10px or 2rem or 10%

right : 10px or 2rem or 10%

bottom : 10px or 2rem or 10%

left : 10px or 2rem or 10%

z-index : 1 or 10 or 100

Note : z-index is responsible for layers arrangement (up & down any layer or element).

10. CSS Flexbox

Flexbox is a CSS layout system used to arrange elements inside a container in a row or column. It makes alignment, spacing and responsive layouts easier.

```
<div class="container">
  <div class="card">Item 1</div>
  <div class="card">Item 2</div>
  <div class="card">Item 3</div>
</div>
```

CSS Flexbox Explain

```
.container{
  display: flex;
  flex-direction: row or row-reverse or column or column-reverse;
  justify-content: value-below;
  align-items: value-below;
  gap: 20px; or row-gap: 20px; or column-gap: 30px;
  flex-wrap: wrap or nowrap or wrap-reverse;
}
```

flex-direction decides the main direction in which flex items are arranged.

row	—	Left to right
row-reverse	—	Right to left
column	—	Top to bottom
column-reverse	—	Bottom to top

justify-content controls how flex items are distributed along the main axis.

flex-start	—	Items start from the beginning
center	—	Items stay in the centre
flex-end	—	Items move to the end
space-between	—	Equal space between items
space-around	—	Space around each item
space-evenly	—	Equal space between and around items

align-items controls the alignment of flex items along the cross axis.

stretch	—	Items stretch by default
flex-start	—	Items move to the start
center	—	Items move to the centre
flex-end	—	Items move to the end

gap is usually cleaner than adding margins to every child just to create spacing.

flex-wrap — By default, flex items try to stay on one line. flex-wrap controls whether they are allowed to move to another line.

nowrap	—	All items stay on one line
wrap	—	Items move to the next line when required
wrap-reverse	—	Items wrap in the opposite cross-axis direction

Note — This property is applied to the **child**.

```
.card { flex-grow: 0 or 1 or 2;}
```

flex-grow controls how much a flex item can grow and take available extra space.

11. CSS Grid

CSS Grid is a two-dimensional CSS layout system used to arrange elements in rows and columns. It is especially useful for creating page layouts, card grids, galleries and other structured layouts.

```
<div class="container">
  <div>Item 1</div>
  <div>Item 2</div>
  <div>Item 3</div>
  <div>Item 4</div>
</div>
```

Here, `.container` is the Grid Container (Parent) and its direct `<div>` elements are Grid Items (Child).

Important: Grid properties are mainly divided into two levels: the **parent controls the grid structure**, while the **child can control its own position or how many rows/columns it occupies**.

CSS Grid Explain

```
.container{
  display: grid;
  grid-template-columns: auto auto auto auto or repeat(4, 1fr) or 100px;
  grid-template-rows: auto auto auto auto or repeat(4, 1fr) or 100px;
  justify-items: start or center or end;
  align-items: start or center or end;
  gap: 20px; or row-gap: 20px; or column-gap: 30px;
  justify-content : start or center or end or space-between or space-around or space-evenly
  align-content : start or center or end or space-between or space-around or space-evenly
}
```

grid-template-columns define the numbers and their width of columns in the grid. (no. of given values define how many columns will be create)

`auto auto` — column occupy space automatically to exact size of parent div. (divide equally no. of given in value "auto")

`100px 200px 100px` — divide proper size of every column whatever it will be given sequentially.

`repeat(3, 1fr) or repeat(4, auto)` — divide or repeat no. of columns in (`_`) equally. equal fraction or auto.

`1fr 2fr 1fr` — divide all columns in specific no. of fractions of parent div. and ratio will be (25% 50% 25%)

12. CSS Media Queries — Responsive Design

What is @media?

@media is used to apply different CSS rules at different screen sizes or conditions. It helps us make a website responsive, so the layout works properly on:

Mobile – Tablet – Laptop – Desktop

max-width means:

Apply these styles when the screen is equal to or smaller than the given width.

min-width means:

Apply these styles when the screen is equal to or larger than the given width.

```
480px /* Small mobile */  
768px /* Tablet */  
1024px /* Laptop / small desktop */  
1280px /* Desktop */
```

// For small mobile screens

```
@media (max-width: 480px) {  
  .box {  
    width: 60%;  
    background: blue;  
  }  
}
```

Means: When the screen width is 480px or smaller, these CSS properties will apply to .box.

// For tablet screens

```
@media (min-width: 481px) and (max-width: 767px) {  
  .box {  
    width: 50%;  
  }  
}
```

Means: These properties apply when the screen width is between 481px and 767px.

// For laptop screens

```
@media (min-width: 768px) and (max-width: 1024px) {  
  .box {  
    width: 50%;  
  }  
}
```

Means: These properties apply when the screen width is 768px to 1024px.

BETTER APPROACH IS MOBILE FIRST APPROACH:

```
/* Default: Mobile */
```

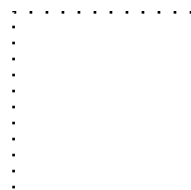
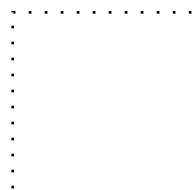
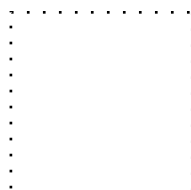
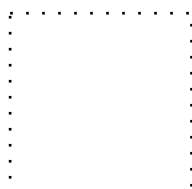
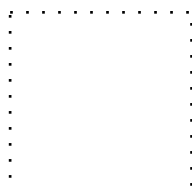
```
.box {  
  width: 100%;  
}
```

```
/* Tablet and above */
```

```
@media (min-width: 768px) {  
  .box {  
    width: 50%;  
  }  
}
```

```
/* Desktop and above */
```

```
@media (min-width: 1024px) {  
  .box {  
    width: 40%;  
  }  
}
```



13. CSS ANIMATIONS

CSS Animation is used to make an HTML element change its style automatically over a period of time. It can be used to create effects such as moving, fading, rotating, scaling and sliding elements.

CSS Animation mainly works with two parts:

1. @keyframes — Defines what changes should happen. (It defines the different stages of an animation.)

2. animation properties — Define how, when and how many times the animation should run.

```
@keyframes move {  
  from {  
    left: 0;  
  }  
  
  to {  
    left: 200px;  
  }  
}
```

```
-----  
@keyframes move {  
  0% {  
    transform: translateX(0);  
  }  
  
  50% {  
    transform: translateX(200px);  
  }  
  
  100% {  
    transform: translateX(0);  
  }  
}
```

```
-----  
.box {  
  animation-name: move;  
  animation-duration: 2s;  
  animation-iteration-count: infinite; }
```